

Chapter 1

Qubicsoft development

Contents

1	Qubicsoft development	1
1.1	Instrumental model	4
1.2	qubicsoft framework	6
1.2.1	Software Architecture - Instrument Simulation	7
1.2.2	Software Architecture - Frequency Map-Making	8
1.2.3	Software Architecture - Component Map-Making	10
1.2.4	Advanced features	13
1.2.5	Additional improvements	16

Introduction

The previous two chapters (Chapters 5 and 6) introduced the FMM and CMM algorithms, together with the simulation framework used throughout this thesis. In particular, both approaches rely on the acquisition operator H , which models the instrument response.

The purpose of this chapter is to present this element in greater detail, describing the construction of the acquisition operator, and the software infrastructure used for simulations and data reconstruction. These developments are implemented within the QUBIC simulation and analysis package, *qubicsoft*¹. Particular emphasis is placed on the architecture of *qubicsoft* and on the design of the simulation, reconstruction, and analysis pipeline. The development of a user-friendly framework integrating these different stages constitutes a major part of the work carried out during this thesis. The chapter also presents the implemented simulation and reconstruction capabilities, discusses the currently known issues and limitations, and outlines the most important developments planned for future releases of the framework.

Personal contribution: At the beginning of this thesis, the simulation and reconstruction pipeline was still at an R&D stage, with many hard-coded or “work-in-progress” components, and was maintained in a separate repository from *qubicsoft*. In collaboration with Mathias Regnier, we improved the software structure of the pipeline by introducing a user-friendly interface based on parameter files and simple Bash² scripts, while migrating the codebase into *qubicsoft*. A significant part of the second

¹<https://github.com/qubicsoft/qubic/tree/master>

²<https://doc.ubuntu-fr.org/bash>

and third years of this thesis was therefore dedicated to software engineering tasks, including refactoring, cleaning, documenting, debugging and optimising the code. These developments were carried out in parallel with hyperparameter characterisation efforts, which are discussed in the next Chapter 9. This work resulted in approximately 650 commits to the *qubisoft* GitHub repository over this period at the time of writing.

1.1 Instrumental model

The instrumental model, denoted by H and extensively used in Chapters 5 and 6, is a complex numerical object based on the work of Mikhail Stolpovskiy during his PhD thesis [1], based on a first mono-frequency implementation from Pierre Chanial. It is constructed as a sequence of operators, each describing a specific stage of the instrument response. These operators are implemented using the PyOperators package [2]³, which provides numerical operators and solvers for high-performance computing, with optimised memory management and efficient computations. Building upon PyOperators, *qubisoft* also relies on the PySimulators package⁴, which provides tools for instrument modelling and map manipulation.

The complete monochromatic acquisition operator used to model the QUBIC instrument can be written as :

$$\mathcal{H}_\nu = \mathcal{R}_{\text{esp}} \mathcal{T}_{\text{trans}} \mathcal{I}_{\text{det}} \mathcal{P}_{\text{ol}} [\mathcal{H}_{\text{WP}} * \mathcal{P}_{\text{roj},\nu}] \mathcal{F}_{\text{ilt}} \mathcal{A}_{\text{perture}} \mathcal{T}_{\text{atm}} \mathcal{U}_{\text{nit},\nu}, \quad (1.1)$$

where each operator represents a specific physical or instrumental effect. Their individual roles are detailed in the following.

Unit Conversion Operator

The Unit Conversion Operator converts the sky signal from HEALPix units, expressed in μK_{CMB} , into spectral flux density unit : $\text{W}/\text{m}^2/\text{Hz}$. The conversion factor is given by :

$$\mathcal{U}_{\text{nit},\nu} = \frac{10^{-6}}{T_{\text{CMB}}} \frac{2\Omega_{\text{pix}} h\nu^3}{c^2} \frac{h\nu}{kT} \frac{e^{\frac{h\nu}{kT}}}{(e^{\frac{h\nu}{kT}} - 1)^2}, \quad (1.2)$$

where Ω_{pix} is the solid angle of a HEALPix pixel and T_{CMB} is the mean temperature of the CMB.

Atmosphere Operator

The Atmosphere Operator models the atmospheric transmission. Throughout the end-to-end simulations presented in this thesis, the atmospheric transmission is assumed to be constant in time. The impact of atmospheric effects and their mitigation are investigated in the dedicated Chapter 11.

³<https://github.com/pchanial/pyoperators>

⁴<https://github.com/pchanial/pysimulators>

Aperture Operator

The Aperture Operator integrates the incoming spectral flux density over the telescope aperture, converting units from $\text{W}/\text{m}^2/\text{Hz}$ to W/Hz . It is represented by a scalar multiplication of value $N_{\text{horn}} S_{\text{horn}} = N_{\text{horn}} \pi r_{\text{horn}}^2$, where N_{horn} is the number of horns (400 for the Full Instrument and 64 for the Technical Demonstrator), S_{horn} is the area of a horn aperture, and r_{horn} is the horn radius.

Filter Operator

The Filter Operator integrates the signal over the considered frequency bandwidth. This bandwidth may be smaller than the physical bandwidth of the instrument when spectral imaging is used (see Chapter 11). The operation converts units from W/Hz to W through integration over the frequency interval.

Projection Operator

The Projection Operator is the central operator of $\mathcal{H}_\nu$, and also the most complicated one. It consists of a coordinate rotation between the instrument frame to the galactic one, which is used to compute the position and amplitude of each peak of the synthesized beam for each detector and for each time sample. The input shape of this operator is that of the HEALPix map $(N_{\text{pix}}, N_{\text{stk}})$, with $N_{\text{stk}} = 3$, the Stokes parameters, and returns TOD with dimension $(N_{\text{det}}, N_{\text{pointings}}, N_{\text{stk}})$. A detailed description of this operator can be found in an internal document written by Thibaut Louis.⁵

This operator is responsible for the memory needed by the simulation algorithm, described in subsection 5.3.3, as well as the majority of the computation time. For these reasons, this part of the code relies on Fortran⁶, to handle properly memory and improve efficiency through multiprocessing via OpenMP.

HWP Operator

The Half-Wave Plate operator applies a rotation of angle φ for each pointing position to the polarisation components, corresponding to the Q and U maps. It again uses Fortran routines to efficiently perform the matrix product :

$$\begin{bmatrix} \tilde{I}^{\text{det}}(t) \\ \tilde{Q}^{\text{det}}(t) \\ \tilde{U}^{\text{det}}(t) \end{bmatrix} = \mathcal{H}_{\text{WP}}(t) \begin{bmatrix} I^{\text{det}}(t) \\ Q^{\text{det}}(t) \\ U^{\text{det}}(t) \end{bmatrix}, \quad (1.3)$$

with :

$$\mathcal{H}_{\text{WP}}(t) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos 4\varphi(t) & \sin 4\varphi(t) \\ 0 & -\sin 4\varphi(t) & \cos 4\varphi(t) \end{pmatrix}. \quad (1.4)$$

⁵https://github.com/qubicsoft/qubic/blob/master/qubic/doc/doc_pdf/doc_QUBICsoft.pdf

⁶With Pierre Chanial, the author and maintainer of PyOperators and PySimulators, we moved the Fortran part from the qubicsoft to PyOperators, to simplify the installation and compilation process for QUBIC software.

Polariser Operator

The Polariser operator selects a polarisation component. It takes a $(N_{\text{det}}, N_{\text{pointings}}, N_{\text{stk}})$ array and project it to a $(N_{\text{det}}, N_{\text{pointings}})$ array by selecting a polarisation component, essentially :

$$\mathcal{P}_{\text{ol}}^{\text{det}}(t) = 0.5 \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}. \quad (1.5)$$

In this convention, it only selects the $\tilde{Q}^{\text{det}}(t)$ component. The resulting signal is : $S(t) = \frac{1}{2} (I(t) + Q(t) \cos(4\varphi(t)) + U(t) \sin(4\varphi(t)))$.

Detector Integration Operator

The Detector Integration operator integrates the flux in the detector solid angles, accounting for the secondary beam profile (described in Chapter 4). Such operator is defined by :

$$\mathcal{I}_{\text{det}} = \frac{\Omega_{\text{det}}}{\Omega_{\text{beam}}} B^{\text{sec}}(\phi_{\text{det}}, \theta_{\text{det}}). \quad (1.6)$$

The solid angle of the detectors is $\Omega_{\text{det}} = \frac{A_{\text{det}}}{D_f^2} \cos^3(\theta_{\text{det}})$, Ω_{beam} the solid angle of the secondary beam, and $B^{\text{sec}}(\phi_{\text{det}}, \theta_{\text{det}})$ is the beam amplitude associated with the detector pointing angles.

Instrument Transmission Operator

The Instrument Transmission operator applies a multiplication by the cumulative instrumental transmission, which is the product of the transmission of all the optical layers :

$$\mathcal{T}_{\text{rans}} = \rho_{\text{det}} \prod_{i=1}^{N_{\text{opt}}} \eta_i, \quad (1.7)$$

with ρ_{det} is the detector efficiency and η_i the transmission of the optical element i .

Detector Response Operator

The Detector Response operator accounts for the time constant introduced by the bolometers (see 2.3.6), of the order ~ 0.01 sec, by convolving the signal by a truncated exponential.

1.2 quicsoft framework

In this section, we aim at defining the software architecture of the simulation and reconstruction pipeline. It is followed by a description of the available features, and the not-working or missing features.

1.2.1 Software Architecture - Instrument Simulation

qubisoft is written mainly in Python ⁷, with some key parts written in Fortran ⁸ for computation efficiency. The main advantages of Python are to be a simple language to use and learn, as well as being the majority within the scientific community. It is an object-oriented programming language, which is a feature widely used across *qubisoft* for its practicality, as well as its ability to simplify the overall organisation of the code.

Fundamental classes

The fundamental class in the *qubisoft* is **QubicInstrument**, which defines all the physical elements of QUBIC instrument : detectors, horns, optics, beams, filters. It includes the methods to build all the operators described in the previous section, as well as the instrumental and photon noise properties. The class only depends on a parameter file defining all the instrumental properties of QUBIC, allowing easy modification of characteristics (to simulate either the TD or the FI for example). **QubicInstrument** is a subclass of **Instrument** from PySimulators.

Then, **QubicAcquisition**, a subclass of **Acquisition** from PySimulators, combines an instrument, a scanning strategy and a sky scene to assemble the instrument's individual operators into the full measurement chain. The operator $\mathcal{H}_\nu$ is built from this class.

These two objects form the core of the simulation and reconstruction pipeline, but are limited to a single frequency range. We denote them as the monochromatic instrument and the monochromatic acquisition operator.

Polychromatic classes

To describe the frequency capabilities of QUBIC, in particular regarding its beam, we define the **QubicMultibandInstrument**. This class holds an array of **QubicInstrument** instances, one per frequency sub-band. It also includes methods to define properly the bandwidth of each of them.

Based on this extension, **QubicMultiAcquisitions** builds one **QubicAcquisition** object per sub-band from **QubicMultibandInstrument**. This class builds the multi-frequency operator H .

Map-Making classes

As discussed during the map-making chapters (5 and 6), we explored the addition of external data during reconstruction. For the addition of Planck, we created the class **PlanckAcquisition**, which contains methods to build the acquisition operator, the noise and the inverse covariance matrix N^{-1} for Planck.

Additionally, we defined the class **QubicInstrumentType**, a subclass of **QubicMultiAcquisitions**. Its purpose is to handle the various instrumental designs of QUBIC (Ultra Wide Band, Dual Bands and Mono Band), which necessitate extra care regarding the shape of the operators.

⁷<https://www.python.org/>

⁸<https://fortran-lang.org/>

Finally, these two classes are joined in two classes : `JointAcquisitionFrequencyMapMaking` and `JointAcquisitionComponentsMapMaking`, which build all the operators, based on their subclasses, for the FMM and CMM.

The complete software architecture of the qubicsoft’s instrument simulation is summarised in Figure 1.1

Remark: An important remark can be made regarding this architecture. At the beginning of this thesis, the software was still at an R&D stage, meaning that many methods had been implemented without an overarching design perspective. As a result, the codebase became highly complex, with numerous special cases and nearly duplicated code to handle the various simulation configurations (instrumental designs, reconstruction methods, sky models, etc.). A significant part of the software development effort that led to the present architecture was therefore devoted to simplifying and generalising the existing structure, while reducing code duplication and eliminating the need for multiple disjoint implementations.

1.2.2 Software Architecture - Frequency Map-Making

Within this thesis framework, the pipeline to use the FMM has been fundamentally simplified. It only necessitates modifying a parameters file, in YAML⁹ format, to describe the simulation. Then, the simulation and reconstruction can be run with only one command line. The configuration file allows setting the physical context of the simulation (composition of the sky, instrumental configuration), as well as the numerical parameters (number of subbands, number of iterations, ...). The FMM is mainly written within one file, where the simulation, the reconstruction, and the computation of the angular power spectra (if asked, see Chapter 7) are performed.

Algorithm

The architecture of the FMM, summarised in Figure 1.2, is the following. First, the parameters file is read and checked for wrong values, to avoid un-implemented cases (for example, wrong instrument configuration), or inconsistent parameters, (for example, if the number of integration sub-bands N_{sub} is not a multiple of the number of reconstructed sub-bands N_{rec}). This prevents running the code with parameters out of its scope, and gives a message to user with feedback of the issue, instead of a raw Python error. If the tests pass, all the necessary objects and variables are created, based on the parameters file. In particular, the class `JointAcquisitionFrequencyMapMaking` is built, which is used to define all the parameters. In the code, a distinction is made between the simulation and the reconstruction, the two not sharing the same parameters to improve realism (this case is discussed in detail in Chapter 9). The input maps are also created, based on PySM3 [3], using a wrapper shared between FMM and CMM. Once all necessary objects initialised, the simulated TOD is created, and the PCG algorithm is used to reconstruct the associated frequency maps. If asked by the user, the cross angular power spectra of the reconstructed maps are computed after the map-making process. The code to compute cross-spectra is designed to run

⁹<https://yaml.org/>

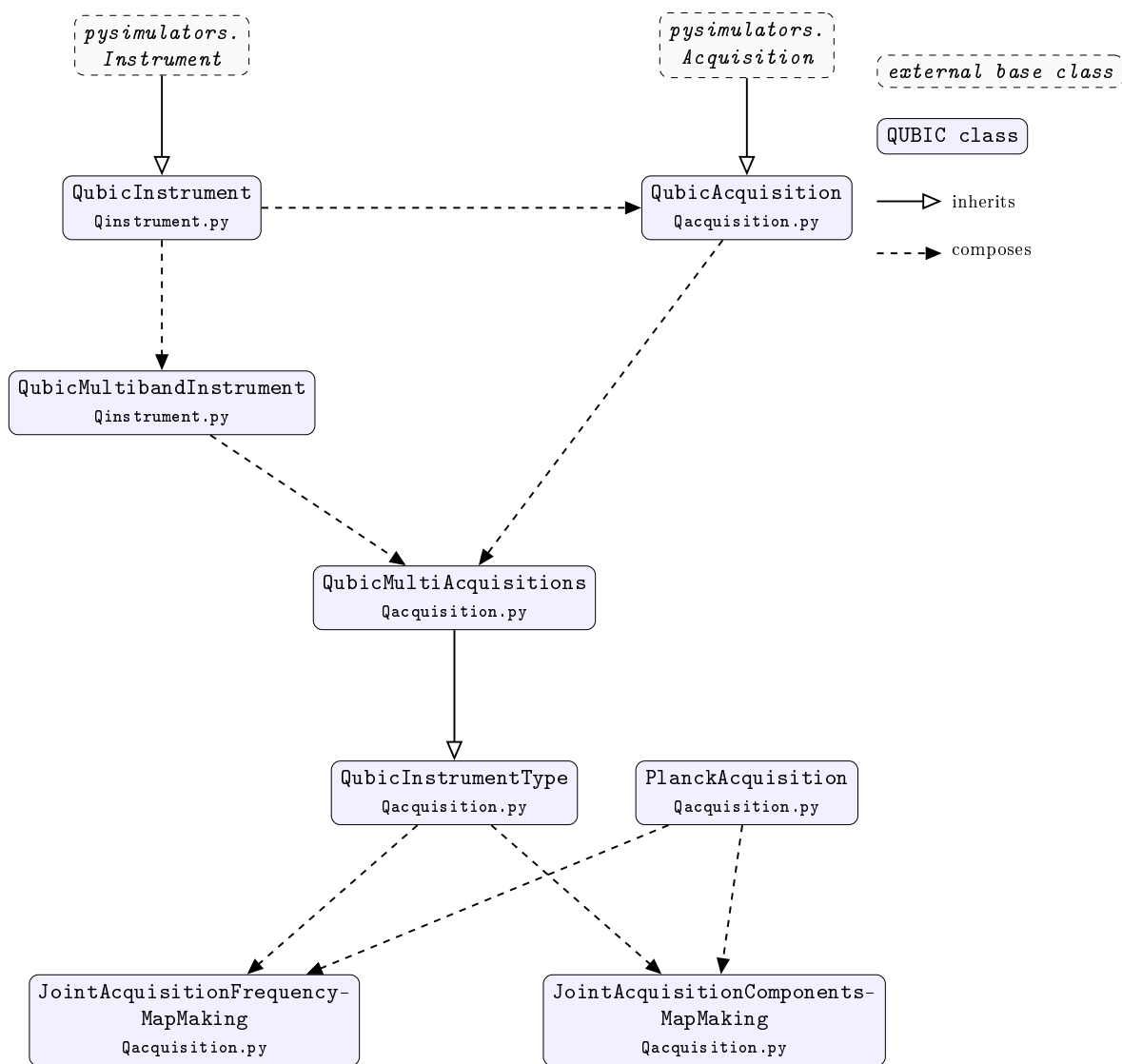


Figure 1.1: Diagram of the software architecture for the simulation of the QUBIC instrument in qubicsft.

for both FMM and CMM. All the relevant information is stored using HDF5¹⁰ format, including TOD, parameters, input and reconstructed maps, coverage map, resolutions, noise and sky cross-spectra with their associated parameters. Automated plots are also run at the same time, to allow quick visualisation of the reconstruction results.

Finally, a dedicated script can be run separately to perform a fit through MCMC on the reconstruction results, to estimate the cosmological parameters such as the tensor-to-scalar ratio r . This script necessitates multiple noise realisations of the simulation, as described in Chapter 7, to run.

1.2.3 Software Architecture - Component Map-Making

The CMM is very similar to the FMM algorithm, and therefore, shares the same general steps. But, its programming complexity is much higher, mainly because the component separation is done alongside the maps' reconstruction. CMM holds the exact same parameters as FMM, but with additional ones: to describe the foreground models, to describe the fitting of the mixing matrix (Parametric or Blind reconstruction) and to handle the alternate fitting. The result is a much more complicated algorithm, in particular regarding the mixing matrix. Therefore, a different architecture has been adopted for the CMM.

First, the check of parameters and the initialisation of all the required objects and variables is done in a series of 8 files, called **Preset**, as summarised in Figure 1.3:

- **PresetTools** : reads and checks the parameters file and initialises the MPI tools.
- **PresetExternal** : selects the Planck frequency bands used as external data.
- **PresetQubic** : builds the QUBIC dictionary and the **JointAcquisitionComponentsMapMaking** object, for both simulation and reconstruction.
- **PresetComponents** : sets the sky configuration and builds the component models and true maps, based on PySM3.
- **PresetSky** : defines the patch center, the coverage, the seen pixels, and the NaMaster objects.
- **PresetGain** : initialises the input and iterated detector gains (optional).
- **PresetMixingMatrix** : initialises the mixing matrix A .
- **PresetAcquisition** : builds the noise covariance and the preconditioner and simulates the TOD. This last preset combines all the previous preset objects.

After the initialisation through these files, all quantities are accessible through the object-oriented architecture across the pipeline.

The algorithm steps of the CMM are very similar to the FMM ones. The main difference stands with the alternate fitting of the mixing matrix and component maps. This extra operation is done within a loop, where both are fitted. We found empirically that the best way to quickly reconstruct the mixing matrix is to perform a lot

¹⁰<https://www.hdfgroup.org/solutions/hdf5/>

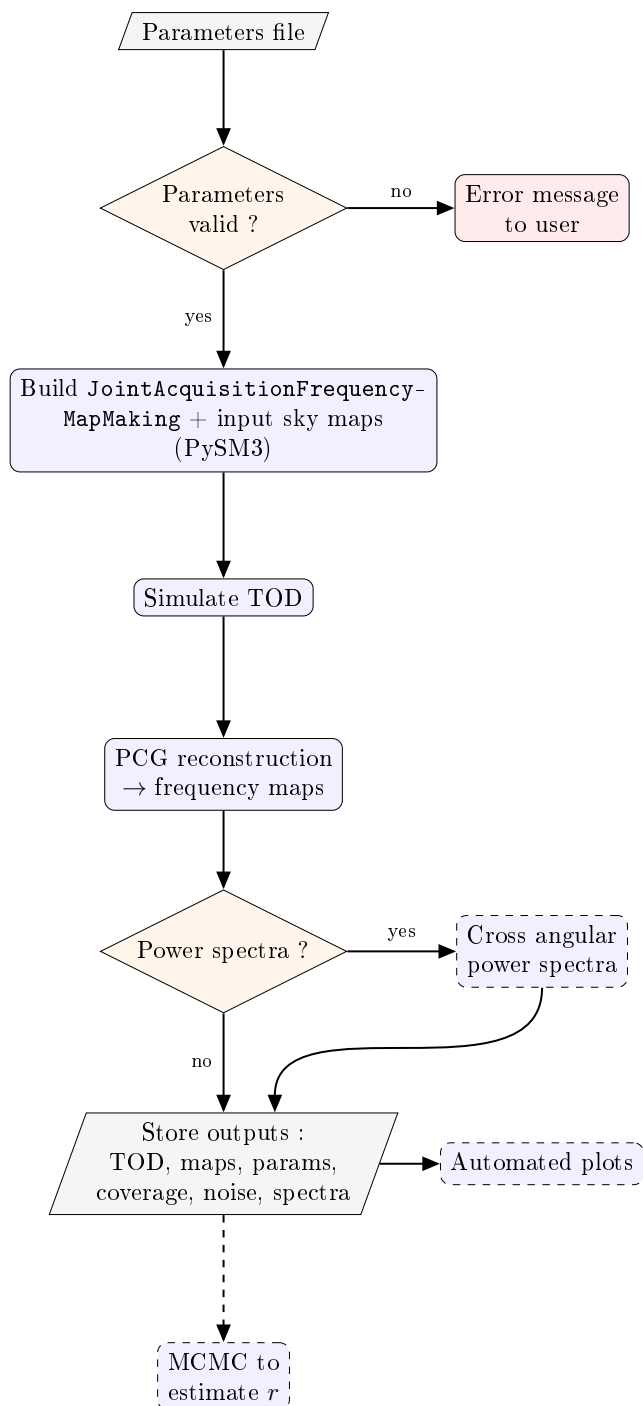
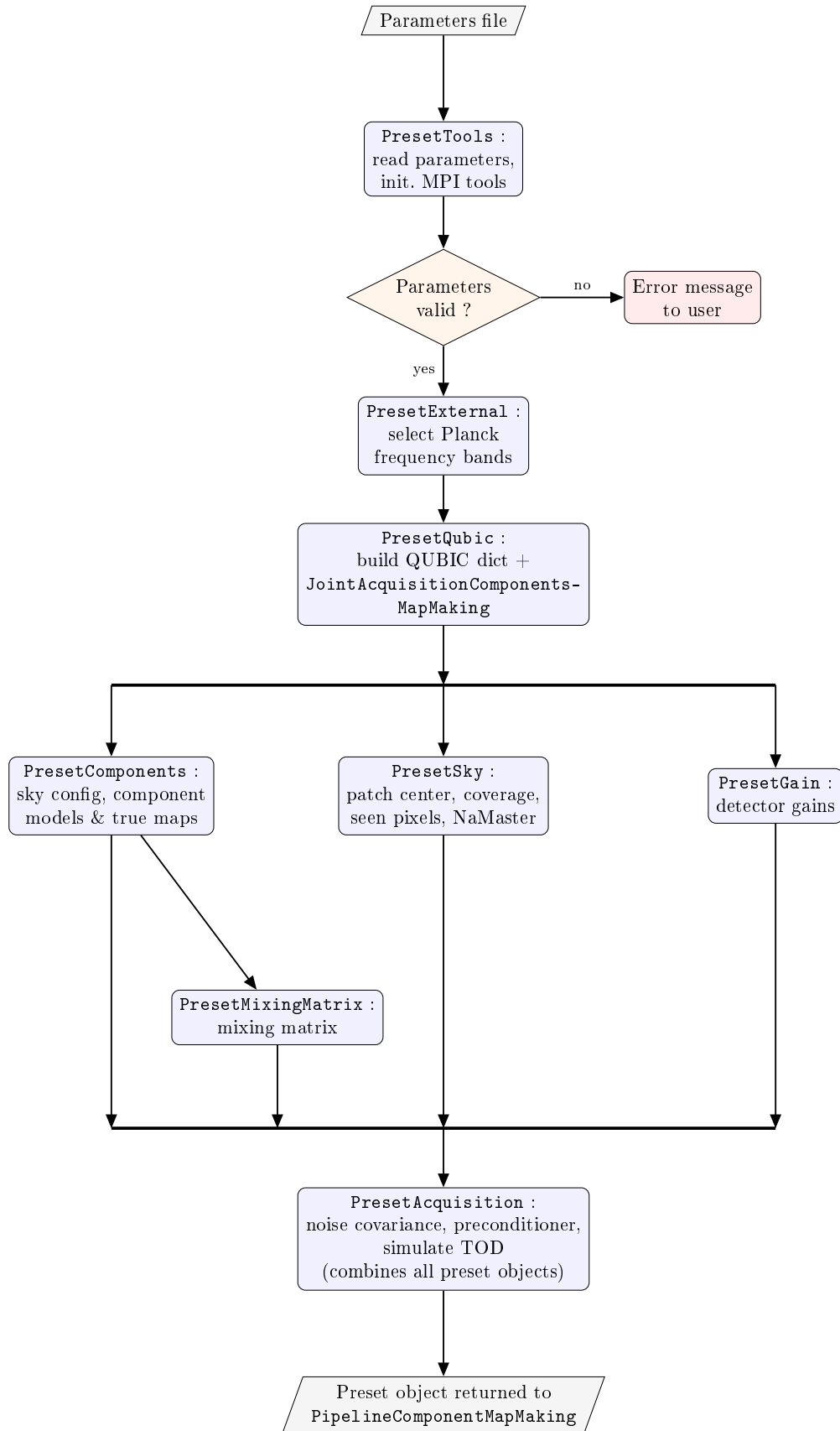


Figure 1.2: Diagram of the FMM pipeline.

Figure 1.3: Diagram of the **Preset** initialisation used by the CMM pipeline.

of iterations within this loop. To keep reasonable computation time, very few PCG iterations to reconstruct component pixels are performed. To ensure correctly reconstructed component maps, we add an extra PCG with a large iterations number once outside the loop. This stage is optional, but still included in the diagram summarising the complete CMM algorithm in Figure 1.4.

Finally, we detail the complicated framework to reconstruct the mixing matrix. The complication comes from the necessity to handle very different foreground models and fitting methods. The Figure 1.5 is summarising the architecture of such algorithm, assuming that the sky is composed only of CMB and Dust. In reality, it also checks for Synchrotron and Carbon Monoxide emission line, going through the same steps for each. For each fitted method, a specific class handles all necessary functions : fitting, updating, plotting and printing methods. **ParametricMM** and **BlindMM** rely on the same general custom class **Chi2** to build the cost function used to fit the mixing-matrix with **scipy.optimize.minimize**¹¹. The Mixed fitting case, written under the class **MixedMM**, is using a specific **MixedChi2** class to build its cost function. It comes from a shape limitation preventing the fit of an array containing a float on one row (parametric case, where one spectral index is fitted) and multiple value on another one (blind case, where multiple mixing matrix elements are fitted). To overcome this issue, an intermediate mapping **MMLayout** is used.

1.2.4 Advanced features

Storage

Previously, simulation outputs were stored using the standard Python Pickle format¹². However, memory limitations were encountered when saving end-to-end simulation results. For this reason, the storage format was changed to HDF5, which is widely used in astrophysics for large-scale data handling. This transition led to a reduction of approximately 60% in memory usage¹³, with increasingly significant gains for larger simulations. To integrate HDF5 within the framework, a wrapper based on the **h5py** package¹⁴ was developed. Indeed, **h5py** does not natively support direct conversion between Python dictionaries containing heterogeneous types (e.g. **numpy arrays**, floats, nested dictionaries, and strings) and HDF5 datasets, both for writing and reading. The implemented wrapper addresses this limitation while preserving reliability and consistency of the data conversion process.

Testing notebooks

A set of testing notebooks was developed to quickly validate simulation outputs. Each notebook targets a specific component of the pipeline: maps, noise, cross-spectra, parameter fitting, and noise realisations, and provides dedicated plotting tools to verify

¹¹Other reconstruction methods have been implemented, but not tested on end-to-end simulations. Therefore, we are not detailing them.

¹²<https://docs.python.org/3/library/pickle.html>

¹³from 587 MB with Pickle to 240 MB with HDF5 for a representative end-to-end simulation

¹⁴<https://pypi.org/project/h5py/>

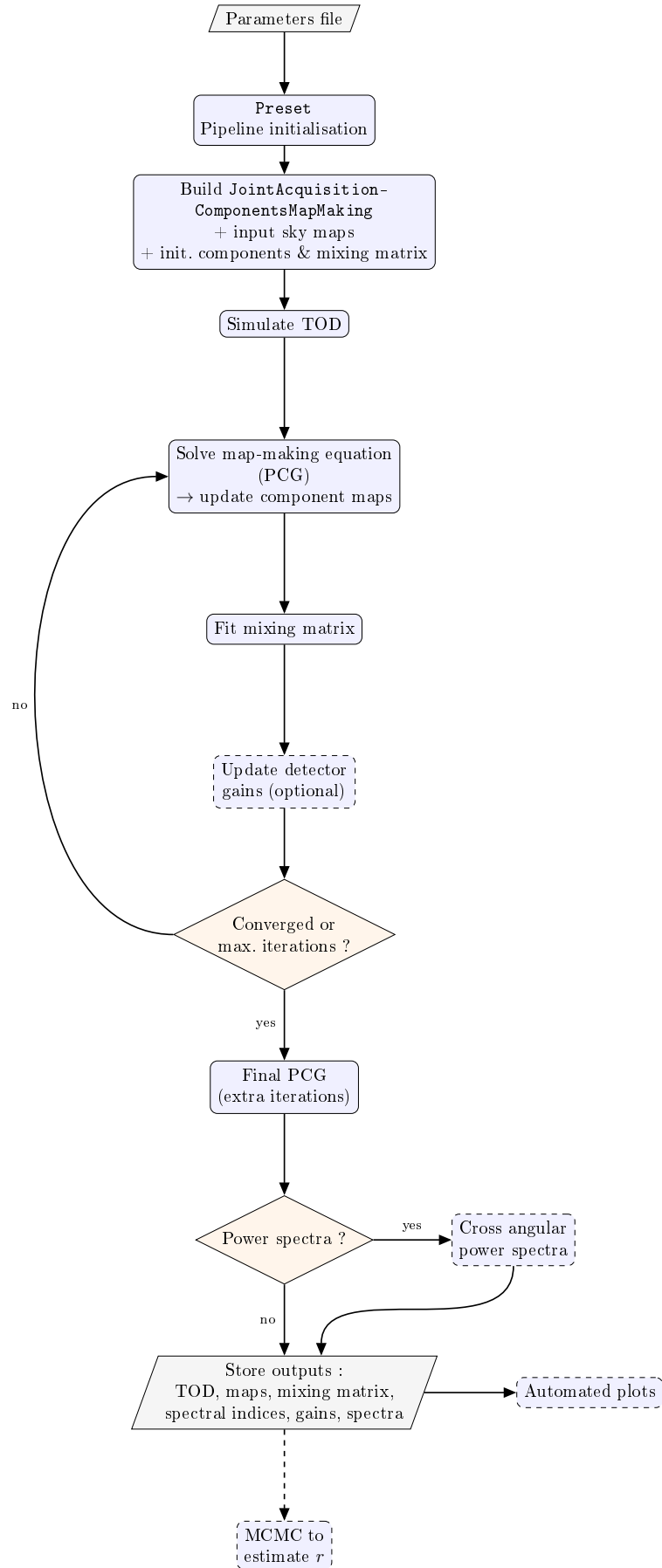


Figure 1.4: Diagram of the CMM pipeline.
– 14 –

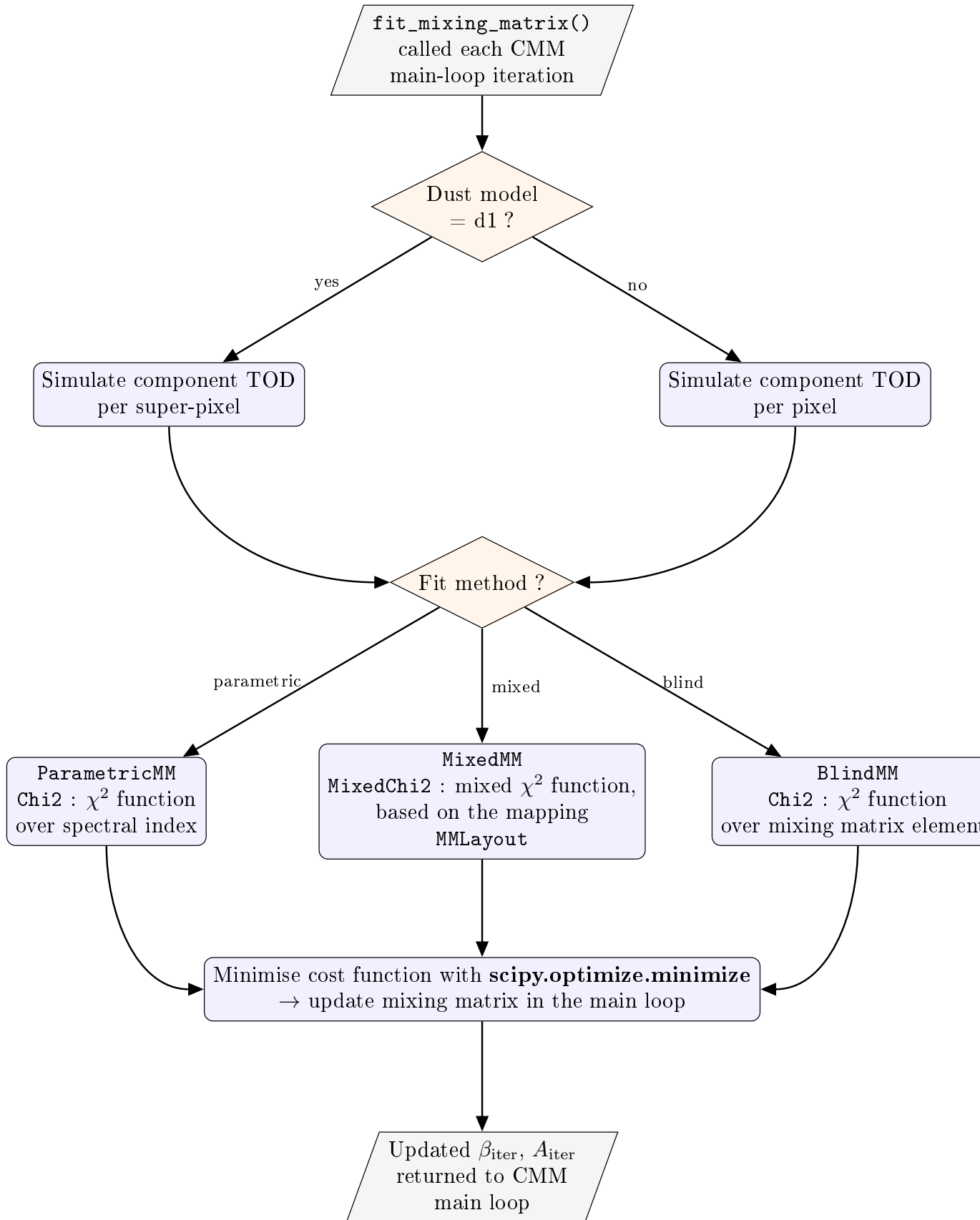


Figure 1.5: Diagram of the mixing-matrix fitting dispatch.

each stage. Most figures presented in this thesis are generated or adapted from these routines.

Computing Center

Given the memory requirements of the map-making algorithms, their computational cost, and the need for a large number of noise realisations, it became necessary to run the pipeline on a computing cluster, in particular the Centre de Calcul de l'IN2P3¹⁵. The code was therefore adapted to this environment by developing dedicated submission scripts and implementing data management tools to handle large-scale parallel simulations.

Map-Making and Cross-Power Spectra

Although map-making reconstruction and cross-spectrum computation are conceptually distinct steps, both were integrated into a single pipeline for simplicity. Computing cross-spectra immediately after each reconstruction avoids introducing an additional post-processing step for the user. Nevertheless, the framework still allows users to perform map-making independently or to compute cross-spectra from pre-existing reconstructions when required.

Simulation and Reconstruction

In addition, since both data simulation and reconstruction are performed within the same pipeline, a functionality was introduced to bypass the simulation step when a time-ordered data (TOD) file is provided by the user. This extension enables two important use cases. First, it constitutes a necessary step towards applying the reconstruction algorithms to real data once the QUBIC instrument becomes operational. Second, from a simulation standpoint, it allows the generation of simulated datasets independently of the reconstruction step.

To support this workflow, a chunk-based simulation strategy was implemented for generating time-ordered data. This approach addresses the primary limitation of full-scale simulations, namely memory usage, which becomes critical for realistic configurations. The TOD is generated iteratively in small chunks, each of manageable size, allowing the construction of large datasets without exceeding memory constraints. Once a simulated TOD is produced, multiple noise realisations can be generated efficiently by adding random noise, avoiding repeated full simulation runs.

This approach enables the use of more realistic simulation parameters while maintaining computational feasibility, and provides a flexible framework to study the impact of different instrumental and analysis choices. This analysis is presented in Chapter 9.

1.2.5 Additional improvements

To conclude this chapter, we briefly discuss additional software improvements made to *qubicsoft*, as well as unfinished or unverified features and remaining developments.

¹⁵<https://cc.in2p3.fr/>

Global refactoring

qubicsoft has been developed over many years by several generations of students and researchers. As a consequence, the codebase accumulated heterogeneous coding practices and varying software engineering standards, motivating a substantial refactoring effort. A significant amount of time was dedicated to cleaning the entire software ($\sim 30\text{k}$ lines of code). This global refactoring includes the adoption of a uniform coding style following PEP8¹⁶, a standard established in 2001 based on the principle that “readability counts”. The PEP8 guidelines are complemented by PEP328¹⁷, which addresses import conventions. To enforce these conventions, a general cleaning of the codebase was performed, including the removal of unused variables and code snippets, the renaming of variables and classes, the addition of comments and docstrings (only in the files I contributed to), and improvements to import statements by importing only the required methods rather than entire modules. This refactoring effort was complemented by the use of automated linters¹⁸ such as Black¹⁹ and isort²⁰, used respectively to enforce code style and sort imports. These tools were later replaced by Ruff²¹, a faster linter combining their functionality.

Automated documentation

An automated documentation system for *qubicsoft* was implemented using Sphinx²². It parses code docstrings to generate a complete documentation, relying on their clarity and completeness. A continuous integration pipeline ensures that the documentation is updated after each pull request on the *qubicsoft* GitHub repository. The documentation is available at: <https://qubicsoft.github.io/qubic/>. No specific effort has yet been made to improve the documentation structure or style; this remains a potential area for future improvement.

Compatibility with Python 3.12

The installation pipeline of *qubicsoft* was initially limited by its dependency on the no-longer-maintained packages PyOperators and PySimulators. As a result, installation was not compatible with recent Python versions (> 3.10). This limitation became increasingly problematic due to version incompatibilities between dependencies used within the codebase. In collaboration with Pierre Chaniel, the maintainer of PyOperators and PySimulators, the installation of both packages was modernised. The new build system now relies on Meson²³ instead of the previous legacy approach.

In addition, the Fortran parts were integrated directly into PySimulators. This modification was motivated by the need to simplify installation for users, as the pre-

¹⁶<https://peps.python.org/pep-0008/>

¹⁷<https://peps.python.org/pep-0328/>

¹⁸A linter is a static analysis tool that automatically detects potential errors, bad practices, and style inconsistencies in the code without executing it.

¹⁹<https://github.com/psf/black>

²⁰<https://github.com/PyCQA/isort/>

²¹<https://github.com/astral-sh/ruff>

²²<https://github.com/sphinx-doc/sphinx>

²³<https://github.com/mesonbuild/meson>

vious setup required compiling Fortran code during installation. This made editable installations difficult and often required full reinstallations after modifications, which was inefficient. The new architecture resolves this issue and provides a more robust and user-friendly installation workflow.

For stability and security reasons, Python 3.12 was not used in this thesis. Although initial tests were successful, further validation is still required. Since these tests were not completed at the time of writing, Python 3.10 was used for all simulations presented.

Unfinished features

Several developments initiated during this thesis could not be completed within the available timeframe and remain topics for future work.

d1 reconstruction: The reconstruction of the d1 dust model (or equivalently, the s1 model for synchrotron emission) corresponds to the case where the spectral index varies spatially across the sky (see Subsection 6.5.1). This specific case is not yet functioning correctly in the CMM algorithm. The code runs and successfully reconstructs all spectral indices across the QUBIC patch, but the fitting procedure diverges when noise is included, even when initialised with the true maps. At the time of writing, it remains unclear whether this issue originates from a bug in the implementation or from an intrinsic limitation in reconstructing spatially varying indices in the presence of noise. Further work is required to clarify this point.

CO reconstruction: The reconstruction of the carbon monoxide emission line (described in Subsection 6.5.2) is not fully implemented. No fundamental limitation prevents its inclusion, and only minimal effort would be required to restore this functionality. However, for prioritisation reasons, this case was deferred to future work, as it is not directly relevant to the primary scientific goal of QUBIC, namely the observation of primordial CMB B-modes.

CMM convolution: The final and most significant unfinished aspect concerns the understanding of the free convolution treatment in CMM. This issue is discussed in Subsection 6.4.2.

Further developments

Several ideas emerged during this thesis that could significantly improve the methods presented here but could not be implemented within the available timeframe.

The most important one is the implementation of a non-linear PCG solver. Such an approach would remove the need for the alternating fitting scheme currently used in CMM and would considerably improve the prospects for atmospheric mitigation discussed in Chapter 11. More importantly, it would provide an optimal convergence to the non-linear nature of the problem, whereas the current alternating fitting procedure can only offer an approximate (and slow) solution. This idea was initially explored by a former internship student, Victor Chabirand, but the limited duration of the internship did not allow for an implementation. We therefore strongly recommend dedicating future development efforts to this topic, as it could represent a major step forward for QUBIC spectral imaging reconstruction.

An alternative to a non-linear PCG approach would be the use of a Gibbs sampling algorithm. Gibbs sampling is a Markov Chain Monte Carlo (MCMC) method

that samples the joint posterior distribution of all model parameters, including the angular power spectra C_ℓ , by iteratively drawing samples from their conditional distributions. This approach is notably employed by the Commander framework [4]. Its main advantage is the rigorous statistical treatment of uncertainties, allowing for automatic marginalisation over systematics while providing full posterior distributions for all quantities of interest, including sky maps, foreground parameters, noise properties, and power spectra. The main drawbacks are the computational cost associated with MCMC methods, their potentially slow convergence, the difficulty of assessing convergence diagnostics, and a substantially more complex implementation compared to the current PCG.

Additionally, the creator of PyOperators and PySimulators, Pierre Chaniel, is working on an improved version of these two packages using JAX²⁴ within the SciPol ERC project²⁵

A more modest improvement concerns FMM and the possibility of reconstructing a different number of sub-bands within the 150 and 220 GHz frequency channels. For example, one could reconstruct several sub-bands within the 220 GHz channel, where thermal dust emission is dominant, while reconstructing only a single map within the 150 GHz channel, where the CMB signal dominates. Such an approach would exploit the spectral imaging capabilities of QUBIC where they are most beneficial for component separation, while limiting the increase in anti-correlated noise associated with a large number of reconstructed sub-bands where little additional information is gained. This idea can be further extended by introducing a non-uniform distribution of sub-band bandwidths. In the current implementation, the bandwidth of each reconstructed sub-band is determined only by the total number of reconstructed maps, N_{rec} , resulting in uniform bandwidths across the frequency range. However, wider sub-bands at lower frequencies and narrower sub-bands at higher frequencies could potentially improve both map reconstruction and component separation by better matching the spectral complexity of the observed sky. A similar strategy could also be applied to the reconstruction of the mixing matrix within the CMM framework.

²⁴<https://github.com/jax-ml/jax>

²⁵<https://scipol.in2p3.fr/>, called FURAX²⁶. The JAX library provides tools for optimising numerical computation (parallelisation across CPU, GPU, and TPU, Just-In-Time compilation, automatic differentiation, and vectorisation, among others), enabling more advanced numerical methods to be applied to large datasets. This framework can highly improve the *qubicsoft*, where computational time and memory usage is always a limitation.